



**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**

**(FIAP)**

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA  
ARTIFICIAL**

**GRUPO SÃO PAULO E INTERIOR**

**JONAS TADEU V. FERNANDES**

**LEVI PASSOS SILVEIRA MARQUES**

**RAPHAEL DA SILVA**

**RAPHAEL DINELLI NETO**

**YAN PIMENTAL COTTA**

**CardiolA:**

**Comunicação automatizada com REST e e-mail**

**SÃO PAULO**

**2026**



**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA  
(FIAP)**

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA  
ARTIFICIAL**

**GRUPO SÃO PAULO E INTERIOR  
JONAS TADEU V. FERNANDES  
LEVI PASSOS SILVEIRA MARQUES  
RAPHAEL DA SILVA  
RAPHAEL DINELLI NETO  
YAN PIMENTAL COTTA**

**CardiolA:**

**Comunicação automatizada com REST e e-mail**

Trabalho acadêmico apresentado ao Curso Superior de Tecnologia em Inteligência Artificial da Faculdade de Informática e Administração Paulista (FIAP), como requisito para obtenção de nota na disciplina AI for Robotic Process Automation.

Tutor: Caique Nonato da Silva Bezerra

**SÃO PAULO**

**2026**



## SUMÁRIO

|                                              |    |
|----------------------------------------------|----|
| 1. INTRODUÇÃO .....                          | 1  |
| 2. DESENVOLVIMENTO .....                     | 2  |
| 2.1 Objetivo do complemento .....            | 2  |
| 2.2 Arquitetura de Aplicação .....           | 2  |
| 2.3 Tecnologias utilizadas .....             | 3  |
| 2.4 Funcionamento do cliente simulador ..... | 4  |
| 2.5 Funcionamento da API REST .....          | 5  |
| 2.6 Lógica de verificação de risco .....     | 5  |
| 2.7 Simulação do envio e-mail .....          | 6  |
| 2.8 Registro dos alertas .....               | 7  |
| 2.9 Boas práticas adotadas .....             | 7  |
| 3. CONCLUSÃO .....                           | 9  |
| FONTES .....                                 | 10 |



## FIGURAS

|                                                  |   |
|--------------------------------------------------|---|
| FIGURA 01 – Fluxo Principal da Arquitetura ..... | 3 |
|--------------------------------------------------|---|



## CÓDIGO

|                                             |   |
|---------------------------------------------|---|
| CÓDIGO 01 – Payload aplicação cliente ..... | 4 |
|---------------------------------------------|---|



## **SIGLAS**

API – Application Programming Interface (Interface de Programação de Aplicações)

REST – Representational State Transfer (Transferência de Estado Representacional)

JSON – JavaScript Object Notation (Notação de Objetos JavaScript)

HTTP – HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)

SMTP – Simple Mail Transfer Protocol (Protocolo Simples de Transferência de Correio)

BPM – Batimentos por Minuto



## RESUMO

O presente relatório apresenta o desenvolvimento complementar da Parte 1 da Fase 3 do projeto Cardiola, com foco na simulação isolada do comportamento do botão de emergência em um sistema de monitoramento cardíaco. Enquanto a Parte 1 concentrou-se na coleta de sinais vitais, armazenamento local e resiliência offline com ESP32, esta etapa complementar utiliza uma API REST em Python para representar o envio e processamento de dados críticos. O cenário simulado considera um paciente com sinais de taquicardia que aciona manualmente o botão de emergência do dispositivo vestível. A aplicação recebe os dados em formato JSON, analisa os riscos com base em regras pré-definidas e simula o disparo automatizado de e-mail em caso de alerta. Foram avaliados parâmetros como frequência cardíaca, temperatura, ausência de movimento e acionamento de emergência. Dessa forma, o complemento demonstra como protocolos HTTP, automação e lógica de risco podem ampliar a resposta do sistema em aplicações de saúde digital.

**Palavras-chave:** API REST; Botão de Emergência; Saúde Digital; Taquicardia; Automação.



## **ABSTRACT**

This report presents the complementary development of Part 1 of Phase 3 of the Cardiola project, focusing on the isolated simulation of the emergency button behavior in a cardiac monitoring system. While Part 1 focused on vital signs acquisition, local storage, and offline resilience using ESP32, this complementary stage uses a Python REST API to represent the transmission and processing of critical data. The simulated scenario considers a patient showing signs of tachycardia who manually presses the emergency button on the wearable device. The application receives data in JSON format, analyzes risks based on predefined rules, and simulates the automated sending of an email in case of an alert. Parameters such as heart rate, temperature, absence of movement, and emergency activation were evaluated. Thus, this complementary implementation demonstrates how HTTP protocols, automation, and risk detection logic can enhance system response in digital health applications.

**Keywords:** REST API; Emergency Button; Digital Health; Tachycardia; Automation.





## 1. INTRODUÇÃO

O projeto CardiolA tem como objetivo simular um sistema vestível de monitoramento cardíaco, integrando conceitos de Internet das Coisas, computação em borda e comunicação em nuvem, visualização de dados e automação aplicada à saúde digital. Na parte 1 da fase 3, o foco principal escreve na implementação do monitoramento local com ESP32, sensores simulados e estratégias de resiliência offline, permitindo que o sistema continuasse coletando e armazenando dados mesmo em ausência de conectividade.

Como complemento dessa etapa, foi desenvolvida uma aplicação separada em Python com o objetivo de demonstrar, de forma isolada, o comportamento do botão de emergência integrado à lógica de análise de risco. Essa separação foi adotada para evidenciar com maior clareza como um evento crítico, originado em um dispositivo vestível, poderia ser recebido por um sistema externo, processado por uma API REST e convertido em uma ação automatização de alerta.

A proposta considerada o seguinte cenário: um paciente monitorado pelo CardiolA apresenta aumento anormal da frequência cardíaca, caracteriza uma possível taquicardia, e aciona manualmente o botão de emergência. A partir desse evento, os dados simulados são enviados para uma API REST, que verifica os sinais vitais recebidos, identifica os riscos existentes e simula o envio de um e-mail automático para um responsável.

Dessa forma, este relatório descreve a arquitetura, o fluxo de funcionamento e a lógica implementada no complemento da Parte 1 da Fase 3, demonstrando como recursos de software podem ampliar a capacidade de resposta de um sistema *IoT* médico.



## **2. DESENVOLVIMENTO**

### **2.1 Objetivo do complemento**

O objetivo deste complemento é simular uma camada externa de comunicação e automação para o projeto CardiolA, utilizando uma API REST desenvolvida em Python. A implementação busca representar o comportamento do botão de emergência quando acionado em uma situação crítica, como taquicardia, ausência de movimento ou outro evento de risco.

Enquanto a parte 1 da fase 3 demonstrou o funcionamento local do dispositivo, este complemento demonstra o fluxo posterior ao evento crítico. Ou seja, a aplicação mostra como os dados poderiam ser enviados para um serviço externo responsável por analisar a situação e acionar uma resposta automatizada.

Assim, o sistema implementado atende aos seguintes objetivos:

- Receber sinais vitais simulados por meio de requisições HTTP;
- Interpretar os dados recebidos em formato JSON;
- Aplicar regras de verificação de risco;
- Identificar eventos como taquicardia, febre, ausência de movimento e botão de emergência pressionado;
- Simular o envio automático de e-mail em caso de alerta;
- Registrar eventos críticos em arquivos de log para consulta posterior.

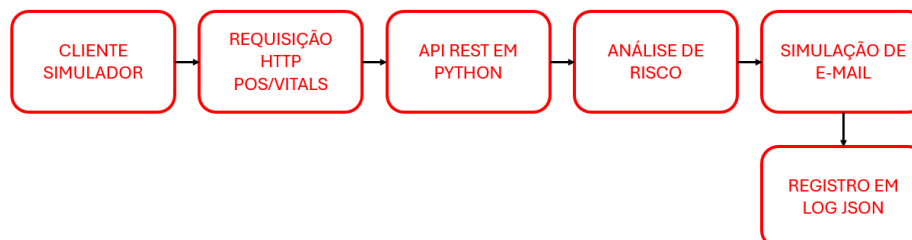
### **2.2 Arquitetura de Aplicação**

A arquitetura do complemento foi organizada em camadas, buscando manter clareza, modularidade e boas práticas de desenvolvimento. O sistema foi dividido entre cliente simulador, API REST, serviço de análise de riscos, serviço de e-mail e serviço de registro de alertas.

O fluxo geral poder ser representado da seguinte forma:



**FIGURA 01 – Fluxo Principal da Arquitetura**



Fonte: Própria Autoria

O cliente simulador representa o dispositivo vestível ou o sistema embarcado enviando dados para a aplicação. A API REST recebe essas informações e as encaminha para serviços de análise de risco. Caso algum alerta seja identificado, o sistema simula o envio de e-mail e registro o evento em arquivos JSON.

Essa arquitetura permite separar responsabilidades. A API fica responsável pela comunicação, o serviço de risco pela tomada de decisão, o serviço de e-mail pela automação e o serviço de *log* pela persistência dos alertas. Essa separação facilita manutenção, testes e futuras expansões.

## 2.3 Tecnologias utilizadas

A implementação foi desenvolvida em Python, utilizando bibliotecas voltadas à construção de APIs, comunicação HTTP e automação de mensagens.

As principais tecnologias utilizadas foram:

- Python:  
Linguagem principal da aplicação
- Flask:  
Framework utilizado para criação da API REST
- Requests:  
Biblioteca utilizada no cliente simulador para envio de requisições HTTP:



- JSON:  
Formato de troca de dados entre cliente e API
- SMTP/smtplib:  
Recurso previsto para envio real de e-mail
- Arquivo JSON local:  
Utilizado para registrar logs dos alertas detectados.

A escolha do *Flask* foi adequada ao escopo do projeto por permitir a criação rápida de *endpoints* HTTP de forma simples e organizada. O uso de JSON como formato de comunicação também é coerente com aplicações REST, pois facilita a troca de informações estruturadas entre sistemas.

## 2.4 Funcionamento do cliente simulador

O cliente simulador foi criado para representar o envio de dados de um dispositivo vestível CardiolA. Nesta implementação, foi considerado um cenário em que o paciente apresenta taquicardia e aciona o botão de emergência.

O payload enviado pela aplicação cliente contém informações como:

### CÓDIGO 01 – Payload aplicação cliente

```
{  
  "device_id": "cardioia_watch_001",  
  "patient_id": "patient_001",  
  "temperature_c": 37.4,  
  "heart_rate_bpm": 132,  
  "movement_detected": false,  
  "emergency_button_pressed": true,  
  "timestamp": "2026-04-26T10:30:00"  
}
```

Fonte: Própria Autoria

Nesse exemplo, o campo *heart\_rate\_bpm* indica uma frequência cardíaca acima do limite configurado. O campo *movement\_detected* indica ausência de



movimento, e o campo *emergency\_button\_pressed* informa que o botão de emergência foi acionado pelo paciente.

Com isso, o cliente envia uma requisição HTTP do tipo *POST* para o *endpoint* */vitals*, simulando a transmissão dos dados para um sistema de monitoramento externo.

## 2.5 Funcionamento da API REST

A API REST foi estruturada para receber os dados enviados pelo cliente, validar a presença do conteúdo JSON e transformar essas informações em um objeto interno da aplicação. Após essa etapa, os dados são encaminhados para o serviço de análise de risco.

Foram definidos dois endpoints principais:

- GET */health*
- POST */vitals*

O endpoint *GET /health* serve para verificar se a *API* está em funcionamento. Já o *endpoint POST /vitals* recebe os sinais simulados e executa o fluxo principal da aplicação.

Quando a *API* recebe uma requisição válida, ela retorna uma resposta contendo o status do processamento, dados recebidos, análise de risco, lista de alertas identificados e informação se o e-mail foi enviado ou simulado.

Esse retorno facilita a verificação do funcionamento do sistema e permite demonstrar claramente o resultado da análise.

## 2.6 Lógica de verificação de risco

A lógica de risco foi construída com regras simples, porém objetiva, coerente com o escopo acadêmico da atividade. O sistema avalia quatro condições principais:

1. Temperatura acima do limite definido;
2. Frequência cardíaca acima de 120 BPM;
3. Ausência de movimento;



#### 4. Botão de emergência pressionado.

A taquicardia é identificada quando a frequência cardíaca recebida ultrapassa o limite configurado de 120 BPM. A ausência de movimento é detectada quando o campo *movimente\_detected* está definido como falso. O botão de emergência é interpretado como evento crítico sempre que o campo *emergency\_button\_pressed* está definido como verdadeiro.

Os alertas são classificados em níveis de severidade, como:

MEDIUM

Alerta moderada, como ausência de movimento

HIGH

Alerta elevado, como taquicardia ou febre

CRITICAL

Alerta crítico, como acionamento do botão de emergência.

A combinação de taquicardia e botão de emergência torna o evento especialmente relevante, pois indica não apenas alteração fisiológica, mas também uma ação consciente do paciente solicitando auxílio.

## 2.7 Simulação do envio e-mail

O enunciado da atividade solicita a simulação de uma automação de envio de e-mail em caso de alerta. Para atender a esse requisito de forma segura e demonstrável, o sistema foi configurado inicialmente em modo de simulação.

Nesse modo, o e-mail não é enviado para uma caixa real, mas seu conteúdo é exibido no terminal. Essa abordagem evita exposição de credenciais, senhas e informações sensíveis durante testes e apresentações; Ainda assim, a estrutura do serviço foi preparada para permitir envio real por SMTP, caso sejam configuradas credenciais adequadas em arquivos de ambiente.

O e-mail simulado contém:

- Identificação do paciente;
- Identificação do dispositivo;
- Data e hora do evento;



- Sinais vitais recebidos;
- Nível geral de risco;
- Lista dos alertas detectados;
- Recomendação de verificação imediata.

Essa estrutura representa um alerta automático funcional e coerente com aplicações de monitoramento em saúde digital.

## **2.8 Registro dos alertas**

Além da simulação do envio de e-mail, os eventos críticos são registrados em um arquivo local no formato JSON. Esse log contém os sinais vitais recebidos, o resultado da análise de risco e a informação se o e-mail foi enviado ou simulado.

O registro dos alertas é importante por três motivos principais, permite auditoria posterior dos eventos, facilita testes e validação do sistema e cria uma base histórica para futura análise de dados.

Em uma aplicação real, esses registros poderiam ser armazenados em um banco de dados, integrados a dashboards ou utilizados por modelos de inteligência artificial para análise preditiva.

## **2.9 Boas práticas adotadas**

O código foi organizado com variáveis em inglês, seguindo uma estrutura clara para facilitar entendimento e manutenção. A aplicação foi dividida em pastas específicas para modelos, rotas, serviços e cliente simulador.

Entre as boas práticas adotadas, destacam-se a separação entre API, análise de risco, e-mail e log. Os nomes das variáveis descritivas, usos de JSON para comunicação, tratamentos básicos de erros e simulação segura do e-mail sem exposição de credenciais. A possibilidade de configuração por variáveis de ambiente e registro dos eventos críticos em arquivo separado.



Essa organização permite que o projeto evolua futuramente pra envio real de e-mail, integração com banco de dados, autenticação de API, dashboards ou conexão com dados enviados pelo ESP32.





### 3. CONCLUSÃO

O complemento desenvolvido para a Parte 1 da Fase 3 do projeto CardiolA permitiu demonstrar, de forma clara e organizada, o comportamento do botão de emergência em uma situação crítica de saúde. A simulação considerou um paciente com taquicardia que aciona manualmente o botão de emergência, gerando um fluxo de comunicação entre cliente, API REST, análise de risco e automação de alerta.

A implementação em Python com Flask mostrou-se adequada ao escopo da atividade, pois permitiu criar uma API simples, funcional e modular. A aplicação recebeu sinais vitais em formato JSON, identificou riscos com base em regras definidas e simulou o envio de e-mail automatizado. Além disso, os alertas foram registrados em arquivo local, reforçando a organização e a rastreabilidade dos eventos.

Esse complemento amplia a proposta inicial da Parte 1, demonstrando que o monitoramento em saúde digital não depende apenas da coleta de dados no dispositivo, mas também de mecanismos externos de resposta e comunicação. A integração entre botão de emergência, API REST e automação de e-mail representa uma etapa importante para construção de sistemas mais seguros, responsivos e aplicáveis a cenários reais.

Conclui-se que a solução implementada atende aos critérios do It Além 1, pois apresenta consumo e envio de dados via API REST, lógica de verificação de risco bem definida, simulação funcional de e-mail automatizado e organização adequada do código. Além disso, reforça a continuidade do projeto CardiolA, conectando conceitos de IoT, saúde digital, automação e resposta a eventos críticos.



## **FONTES**

PYTHON ENHANCEMENT PROPOSALS. PEP 20 - The Zen of Python.

Disponível em: <https://peps.python.org/pep-0020/#the-zen-of-python>. Acesso em 25/04/2026.

PALLETS PROJHECTS. Flask Documentation.

Disponível em: <https://flask.palletsprojects.com/>. Acesso em 25/04/2026.

REQUEST. Requests: HTTP for Humans.

Disponível em: <https://requests.readthedocs.io/>. Acesso em: 25/04/2026.

PYTHON SOFTWARE FOUNDATION. smtplib — SMTP protocol cliente.

Disponível em: <https://docs.python.org/3/library/smtplib.html>. Acesso em: 26/04/2026.

Faculdade de Informática e Administração Paulista (FIAP). REST e e-mail como motores de processos em RPA. São Paulo: 2026.

